American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

PROBLEM: Given a string to create 2 parallel arrays, process the letters in the string from left to right, inserting each letter into the first array in alphabetical order, one letter at a time. During this process, each letter is assigned an integer value in the second array as follows:

1. The first letter has a value of 0.
2. If the new letter is the first or last in the array, its value is one more than the value of the adjacent letter.
3. Otherwise, it is one more than the larger value of the two adjacent values. If the letter is already in the array, the new letter is placed before the existing letter.

Once the two arrays are created, output two different strings separated by a single space.
To create the first string, add letters to the string as they are found starting with the letter that has a value of 0.

1. Look to its left to find a letter with the next consecutive value. If it exists, add it to the string.
2. Stop the recursion if no more letters are available or a letter with a lower value is found.
3. Look to the current letter's right to find a letter with the next consecutive value. If it exists, add it to the string.

Do steps 1-3 recursively for each letter found to the left of the letter with a value of 0.
Find the 1 to the right of the letter with a value of 0.
Do steps 1-3 recursively for each letter found to the right of the letter with a value of 0.

To create the second string, access the letters in the same consecutive order as above looking first to the left and then to the right. Stop the recursion if you find a letter such that nothing that can be found after that. Nothing can be found if either there are no more letters or a letter with a smaller value is found first. Add that letter to the string. Return to the previous letter to continue the recursion.

EXAMPLE #1: Here is how the array is built for the string **PYTHONN**:
The letter P illustrates rule 1; Y and H illustrate rule 2; the T, O and Ns illustrate rule 3; and the final N illustrates what happens if there is a duplicate letter.

P	*Letters*	P								
	Values	0								
Y	*Letters*	P	Y							
	Values	0	1							
T	*Letters*	P	T	Y						

	Values	0	2	1						
H	*Letters*	H	P	T	Y					
	Values	1	0	2	1					
O	*Letters*	H	O	P	T	Y				
	Values	1	2	0	2	1				
N	*Letters*	H	N	O	P	T	Y			
	Values	1	3	2	0	2	1			
N	*Letters*	H	N	N	O	P	T	Y		
	Values	1	4	3	2	0	2	1		

The steps used to find the first and second string are as follows:

- Access the P with a value of 0, then the H with a value of 1 to the left of the P. There are no letters that can be found to the left of H. Find the O to the right of H with a value of 2, then N with a value of 3 to the left of O, then the N with a value of 4 to the left of the N with a value of 3. There are no letters that can be found with a value greater than 4 to the left or the right of the N with a value of 4 so stop the recursion. There are also no values greater than 3 to the right of the N with a value of 3 and no values greater than 2 to the right of the O. To the right of the P, find the Y with a value of 1, and finally the T to its left with a value of 2.

- In the first string, add each letter as it is found. The result is PHONNYT.

- For the second string, after the P, find the H, then the O, then the N with a value 3, then the N with a value of 4. Since there is nothing else to its left or right that can be found, add the N with a value of 4 to the string. Return to the previous letter which is the N with a value of 3 and add it to the string. Return to the O, add it to the string, return to the H, and add it to the string. There are no more letters that can be found to the left of P so look to the right of P. Find the Y, then the T. Therefore, add the T to the string since there are no more letters that can be found to both its left and right, then the Y, and finally the P. The result is NNOHTYP.

The output for these arrays is: PHONNYT NNOHTYP.

EXAMPLE #2: The string **BINARYSEARCHTREE** results in the following array:

Letters	A	A	B	C	E	E	E	H	I	N	R	R	R	S	T	Y
Values	2	1	0	3	5	4	2	3	1	2	5	4	3	5	6	4

Access the letters for both strings as follows: On the left of B with a value of 0, find the A with a value of 1, then the A with a value of 2 and stop. On the right of B with a value of 0, find the I with a value of 1, then the E to the left with a value of 2, then the C to the left with a value of 3, then the E to the right with a value of 4, then the E to the left with a value of 5, then the H with a value of 3 to the right of the E with a value of 2 since you have done everything to the left of the E with a value of 2 and now must look to its right. Continue on the right side of the I with a value of 1. The output for these arrays is: BAAIECEEHNRRRYST AAEECHERRTSYRNIB.

INPUT: Input a string to use in building the two arrays. The string will be no more than 80 characters containing all capital letters.

OUTPUT: Output a string representing the first and second strings separated by a single space.

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

SAMPLE INPUT:

1. PYTHONN
2. BINARYSEARCHTREE
3. CORONAVIRUS
4. FINALSFORACSL
5. HACKERRANKPLATFORM

SAMPLE OUTPUT:

1. PHONNYT NNOHTYP
2. BAAIECEEHNRRRYST AAEECHERRTSYRNIB
3. CAOONIRRVUS AINORSUVROC
4. FAAFCINLLSORS ACFALLSROSNIF
5. HAAACEFKKRRNLMPORT AAFECAKMLORPNRTRKH

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

TEST DATA

TEST INPUT:

1. MOTHER
2. ACSLCONTEST
3. SUPERCALIFRAGILISTIC
4. JAVAPROGRAMMING
5. ABDFHKMOQTVWYZ

TEST OUTPUT:

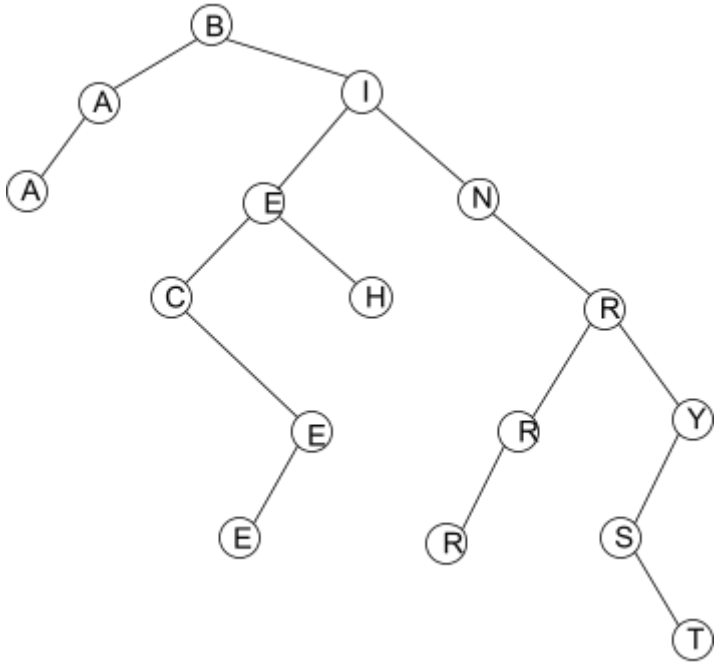
1. MHEOTR EHRTOM
2. ACCSLEONSTT CENSOLTTSCA
3. SPECAACLIFGIIILRRSUT ACACIIIGFLILERSRPTUS
4. JAAAGGIVPOMMNRR AAGIGAMNMORRPVJ
5. ABDFHKMOQTVWYZ ZYWVTQOMKHFDBA

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

FOR INFORMATION PURPOSES ONLY

The procedure above is a way to create a binary search tree. The values in the array represent the depth of the nodes. The tree below is the result:



The letters printed are the preorder and postorder traversals of the tree. The preorder traversal is root→left→right recursively which produces BAAIECEEHNRRRYST. The postorder traversal is left→right→root recursively which produces AAEECHERRTSYRNIB.

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

问题: 给定一个字符串，用于创建 2 个平行数组，从左到右处理字符串中的字母，按字母表顺序将字母依次插入第一个数组，每次一个字母。在此过程中，在第二个数组中为每个字母分配一个整数值，规则如下所示：

1. 第一个字母的数值是 0。
2. 如果新插入的字母是数组中的第一个或最后一个，则它的数值比相邻字母的数值大 1。
3. 否则，它比两个相邻值中的较大值大 1。如果这个字母已经在数组中，则将新插入的字母放在现有字母之前。

创建两个数组后，打印输出由一个空格分隔的两个不同字符串。
要创建第一个字符串，请将以数值为 0 开始的字母添加到字符串中。

1. 搜索这个字母的左侧，找到具有下一个连续数值的字母。如果存在，将其添加到字符串中。
2. 如果没有更多符合条件的字母或找到一个数值较低的字母，则停止递归。
3. 搜索当前字母的右侧，查找具有下一个连续数值的字母。如果存在，将其添加到字符串中。

对在数值为 0 的字母的左侧搜索到的每个字母，递归执行步骤 1-3。
在数值为 0 的字母右侧找到数值为 1 的字母。
对在数值为 0 的字母的右侧搜索到的每个字母，递归执行步骤 1-3。

要创建第二个字符串，按照与上面相同的连续顺序查找字母，先搜索左侧，然后搜索右侧。如果没有更多的字母或先搜索到一个数值较小的字母，说明无法找到任何字母，此时停止递归，并将这个字母添加到字符串中。然后返回前一个字母，继续执行递归。

示例 #1: 下面展示了如何对字符串 **PYTHONN** 创建数组。
字母 P 表示规则 1； Y 和 H 表示规则 2；T、O 和 N 表示规则 3；最后一个 N 表示如果有重复的字母会如何在数组中处理。

P	*字母*	P								
	数值	0								
Y	*字母*	P	Y							
	数值	0	1							
T	*字母*	P	T	Y						

	数值	0	2	1						
H	*字母*	H	P	T	Y					
	数值	1	0	2	1					
O	*字母*	H	O	P	T	Y				
	数值	1	2	0	2	1				
N	*字母*	H	N	O	P	T	Y			
	数值	1	3	2	0	2	1			
N	*字母*	H	N	N	O	P	T	Y		
	数值	1	4	3	2	0	2	1		

搜索第一个和第二个字符串的步骤，如下所示：

- 访问数值为 0 的字母 P，然后在 P 的左侧，找到数值为 1 的 H。在 H 左侧没有其他字母。在 H 的右侧，找到数值为 2 的 O，在 O 的左侧找到数值为 3 的 N，在数值为 3 的 N 的左侧找到数值为 4 的 N。在数值为 4 的 N 的左侧或右侧不能搜索到数值大于 4 的字母，因此停止递归。数值为 3 的 N 的右侧没有大于 3 的数值，O 的右侧没有大于 2 的数值。在 P 的右侧，找到数值为 1 的 Y，最后在它的左侧找到数值为 2 的 T。

- 在第一个字符串中，添加入搜索到的每个字母，得到 PHONNYT.

- 对于第二个字符串，P 之后，搜索到 H，然后是 O，然后搜索到数值为 3 的 N，再找到数值为 4 的 N。由于在它的左侧或右侧找不到其他字母，所以在字符串中添加数值为 4 的字母 N。返回它的前一个字母，即数值为 3 的 N，并将其添加到字符串中。返回到 O，将其添加到字符串，返回到 H，然后将其添加至字符串。没有其它字母可以在 P 的左侧找到，所以请看 P 的右侧。找到 Y，然后是 T。此时因为没有更多字母可以在它的左侧和右侧找到，将 T 添加到字符串中，然后添加 Y 最后添加 P。结果得到 NNOHTYP。

这组数组最终输出为： PHONNYT NNOHTYP.

示例 #2: 字符串 **BINARYSEARCHTREE** 可以得到以下数组：

字母	A	A	B	C	E	E	E	H	I	N	R	R	R	S	T	Y
数值	2	1	0	3	5	4	2	3	1	2	5	4	3	5	6	4

按如下方式访问两个字符串的字母：在数值为 0 的 B 的左侧，找到数值为 1 的 A，然后找到数值为 2 的 A 并停止。在数值为 0 的 B 的右侧，找到数值为 1 的 I，然后找到左侧数值为 2 的 E，然后在左侧找到数值为 3 的 C，然后在右侧找到数值为 4 的 E，然后发现左侧数值为 5 的 E，然后数值为 3 的 H 位于数值为 2 的 E 的右侧，因为对于数值为 2 的 E 的左侧已完成了所有搜索，现在搜索它的右侧。在数值为 1 的 I 的右侧继续搜索。
这组数组的输出为 BAAIECEEHNRRRYST AAEECHERRTSYRNIB。

输入: 输入一个字符串用于创建 2 个数组，这个字符串的所有字母为大写字母，且不超过 80 个字母。

输出: 输出一个字符串表示第一个字符串和第二个字符串，中间用一个空格隔开。

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

样本输入：

1. PYTHONN
2. BINARYSEARCHTREE
3. CORONAVIRUS
4. FINALSFORACSL
5. HACKERRANKPLATFORM

样本输出：

1. PHONNYT NNOHTYP
2. BAAIECEEHNRRRYST AAEECHERRTSYRNIB
3. CAOONIRRVUS AINORSUVROC
4. FAAFCINLLSORS ACFALLSROSNIF
5. HAAACEFKKRRNLMPORT AAFECAKMLORPNRTRKH

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

测试数据

测试输入：

1. MOTHER
2. ACSLCONTEST
3. SUPERCALIFRAGILISTIC
4. JAVAPROGRAMMING
5. ABDFHKMOQTVWYZ

测试输出：

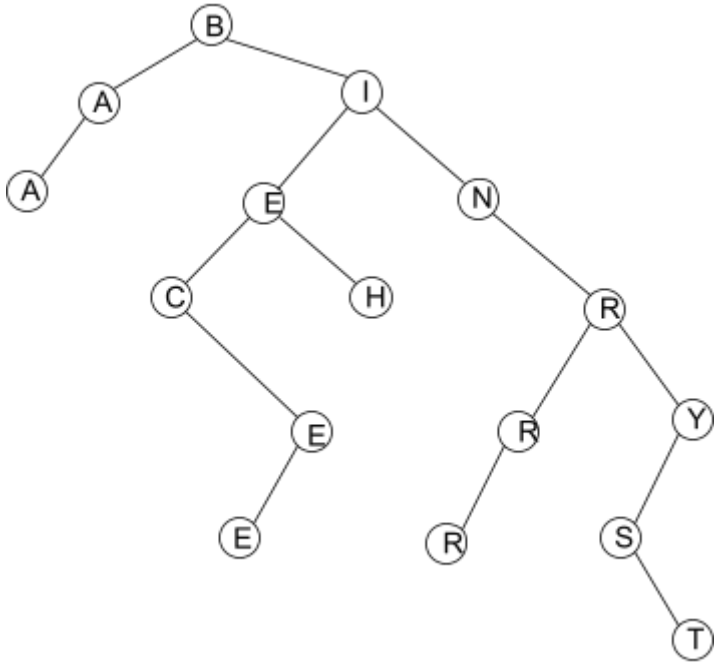
1. MHEOTR EHRTOM
2. ACCSLEONSTT CENSOLTTSCA
3. SPECAACLIFGIIILRRSUT ACACIIIGFLILERSRPTUS
4. JAAAGGIVPOMMNRR AAGIGAMNMORRPVJ
5. ABDFHKMOQTVWYZ ZYWVTQOMKHFDBA

American Computer Science League

2022-2023 ● Contest 3: Create a Tree ● Senior Division

FOR INFORMATION PURPOSES ONLY

The procedure above is a way to create a binary search tree. The values in the array represent the depth of the nodes. The tree below is the result:



The letters printed are the preorder and postorder traversals of the tree. The preorder traversal is root→left→right recursively which produces BAAIECEEHNRRRYST. The postorder traversal is left→right→root recursively which produces AAEECHERRTSYRNIB.